Hostel Cleaning Desk: Producer-Consumer Simulator (hostel-producer-consumer.html)

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1, viewport-fit=cover">
<title>Hostel Cleaning Desk ? Producer?Consumer Simulator</title>
<link href="https://fonts.googleapis.com/css2?family=Bricolage+Grotesque:wght@500;700;800&family=IBM+Plex+Sans:wght@40
      0;500&display=swap" rel="stylesheet">
<style>
:root{--bg:#eceee6;--panel:#f8f8f3;--ink:#1c2b33;--mute:#5d6b72;--line:#cfd4c8;--door:#8c5b3a;--tag:#f0c24b;--
      ok:#2e7d5b;--bad:#bf3f2d;--blue:#2f6690;--blk:#b9772a;}
@media(prefers-color-scheme:dark){:root:not([data-theme="light"]){--bg:#12181c;--panel:#1b242a;--ink:#e6ece8;--
      mute:#98a6ac;--line:#2f3d45;--door:#b98259;--tag:#d9ab36;--ok:#52b389;--bad:#e5705e;--blue:#6aa7d4;--
      blk:#e0a04e;}}
:root[data-theme="dark"]{--bg:#12181c;--panel:#1b242a;--ink:#e6ece8;--mute:#98a6ac;--line:#2f3d45;--door:#b98259;--
      tag:#d9ab36;--ok:#52b389;--bad:#e5705e;--blue:#6aa7d4;--blk:#e0a04e;}
*{box-sizing:border-box}
:root{padding-top:env(safe-area-inset-top,0px);padding-bottom:env(safe-area-inset-bottom,0px)}
html{scroll-padding-top:env(safe-area-inset-top,0px)}
body{margin:0;background:var(--bg);color:var(--ink);font:15px/1.5 "IBM Plex Sans",system-ui,sans-serif}
h1,h2,h3{font-family:"Bricolage Grotesque","Trebuchet MS",sans-serif;margin:0;letter-spacing:-.01em}
.wrap{max-width:1100px;margin:0 auto;padding:20px 16px 40px}
header{display:flex;flex-wrap:wrap;gap:12px;justify-content:space-between;align-items:end;margin-bottom:16px}
h1{font-size:clamp(26px,5vw,40px);font-weight:800;line-height:1.05}
header p{margin:6px 0 0;color:var(--mute);max-width:56ch}
.panel{background:var(--panel);border:1px solid var(--line);border-radius:10px;padding:14px}
.controls{display:grid;grid-template-columns:repeat(auto-fit,minmax(170px,1fr));gap:12px 18px;align-items:end;margin-
      bottom:16px}
.controls label{display:flex;flex-direction:column;font-size:13px;color:var(--mute);gap:4px}
.controls label b{color:var(--ink);font-weight:500}
input[type=range]{width:100%;accent-color:var(--blue)}
.btns{display:flex;flex-wrap:wrap;gap:8px;grid-column:1/-1}
button{font:500 14px "IBM Plex Sans",sans-serif;border:1px solid
      var(--ink);background:transparent;color:var(--ink);padding:8px 14px;border-radius:6px;cursor:pointer}
button.primary{background:var(--ink);color:var(--bg)}
button:hover{opacity:.85}
:focus-visible{outline:3px solid var(--blue);outline-offset:2px}
.mode{display:flex;gap:8px;align-items:center;font-size:14px;color:var(--ink)}
.stage{display:grid;grid-template-columns:1fr 1.5fr 1fr;gap:14px;margin-bottom:16px}
@media(max-width:820px){.stage{grid-template-columns:1fr}}
.col h2{font-size:18px;margin-bottom:2px}
.col .sub{color:var(--mute);font-size:13px;margin-bottom:10px}
.actor{display:flex;justify-content:space-between;gap:8px;align-items:center;border:1px solid var(--line);border-
      left:5px solid var(--mute);border-radius:6px;padding:8px 10px;margin-bottom:8px;background:var(--bg)}
.actor small{display:block;color:var(--mute);font-size:12.5px}
.actor.working{border-left-color:var(--ok)}
.actor.blocked{border-left-color:var(--blk);background:color-mix(in srgb,var(--blk) 12%,var(--bg))}
.actor.cs{border-left-color:var(--blue);background:color-mix(in srgb,var(--blue) 12%,var(--bg))}
.actor.err{border-left-color:var(--bad)}
.chip{font-size:12px;white-space:nowrap;border-radius:99px;padding:2px 9px;background:var(--line);color:var(--ink)}
.corridor{display:grid;grid-template-columns:repeat(auto-fill,minmax(120px,1fr));gap:10px}
.slot{position:relative;border:2px solid var(--door);border-radius:8px 8px 3px 3px;min-
      height:92px;padding:8px;background:var(--bg);display:flex;flex-direction:column;justify-
      content:center;transition:background .25s}
.slot .idx{position:absolute;top:3px;right:6px;font-size:11px;color:var(--mute)}
.slot.full{background:color-mix(in srgb,var(--tag) 22%,var(--bg))}
.slot .tag{background:var(--tag);color:#2a2208;border-radius:3px 12px 3px 12px;padding:5px 8px;font-size:13px;font-
      weight:500;line-height:1.25}
.slot .empty{color:var(--mute);font-size:13px;text-align:center}
.ptr{font-size:11px;font-weight:500;margin-top:5px;display:flex;gap:6px}
.ptr span{padding:0 6px;border-radius:3px;color:#fff}
.ptr .in{background:var(--blue)}.ptr .out{background:var(--ok)}
.bottom{display:grid;grid-template-columns:1fr 1fr;gap:14px;margin-bottom:16px}
@media(max-width:820px){.bottom{grid-template-columns:1fr}}
.sems{display:grid;grid-template-columns:repeat(3,1fr);gap:8px;margin-bottom:10px}
.sem{border:1px solid var(--line);border-radius:8px;padding:8px;text-align:center;background:var(--bg)}
.sem b{display:block;font-family:"Bricolage Grotesque",sans-serif;font-size:28px;line-height:1.1}
.sem span{font-size:12px;color:var(--mute)}
.stats{display:grid;grid-template-columns:repeat(4,1fr);gap:8px}
.stats div{text-align:center;font-size:12px;color:var(--mute)}
.stats b{display:block;font-size:22px;color:var(--ink);font-family:"Bricolage Grotesque",sans-serif}
.stats .bad b{color:var(--bad)}
#log{height:230px;overflow:auto;font-size:13px;margin:0;padding:0;list-style:none}
#log li{padding:3px 0;border-bottom:1px dashed var(--line)}
#log .e{color:var(--bad);font-weight:500}#log .p{color:var(--blue)}#log .c{color:var(--ok)}#log .w{color:var(--blk)}
pre{margin:0;overflow-x:auto;font-size:12.5px;line-height:1.5;background:var(--bg);border:1px solid
      var(--line);border-radius:6px;padding:10px;max-width:100%}
.code2{display:grid;grid-template-columns:1fr 1fr;gap:10px}
@media(max-width:620px){.code2{grid-template-columns:1fr}}
.note{color:var(--mute);font-size:13px;margin:8px 0 0}
@media(prefers-reduced-motion:reduce){*{transition:none!important}}
</style>
</head>
<body>
<div class="wrap">
```

Hostel Cleaning Desk: Producer-Consumer Simulator (hostel-producer-consumer.html)

```
<header>
  <div>
    <h1>Hostel Cleaning Desk</h1>
    <p>Students post room-cleaning requests onto a fixed-size notice board. Housekeeping staff take them off.
      Semaphores keep the board from overflowing, being read empty, or being corrupted by two people at once.</p>
  </div>
  <button id="theme" aria-label="Toggle dark mode">Light / dark</button>
</header>

<section class="panel controls" aria-label="Simulation controls">
  <label>Board capacity: <b id="vN">5</b> requests<input id="N" type="range" min="2" max="8" value="5"></label>
  <label>Students (producers): <b id="vP">3</b><input id="P" type="range" min="1" max="5" value="3"></label>
  <label>Staff (consumers): <b id="vC">2</b><input id="C" type="range" min="1" max="5" value="2"></label>
  <label>Request rate: <b id="vPr">3.0 s</b> per request<input id="Pr" type="range" min="5" max="60"
      value="30"></label>
  <label>Cleaning time: <b id="vCr">4.0 s</b> per room<input id="Cr" type="range" min="5" max="80" value="40"></label>
  <label>Simulation speed: <b id="vSp">1×</b><input id="Sp" type="range" min="1" max="6" value="2"></label>
  <label class="mode"><span><input type="checkbox" id="sync" checked> Use semaphores (synchronised)</span></label>
  <div class="btns">
    <button class="primary" id="go">Start</button>
    <button id="reset">Reset</button>
    <button id="presetFull">Preset: staff too slow</button>
    <button id="presetEmpty">Preset: students too slow</button>
  </div>
</section>

<section class="stage">
  <div class="panel col"><h2>Students</h2><div class="sub">Producers: write and post requests</div><div
      id="prods"></div></div>
  <div class="panel col"><h2>Notice board</h2><div class="sub">Bounded buffer (circular queue)</div><div
      class="corridor" id="board"></div></div>
  <div class="panel col"><h2>Housekeeping</h2><div class="sub">Consumers: take a request, clean the room</div><div
      id="cons"></div></div>
</section>

<section class="bottom">
  <div class="panel">
    <h2>Semaphores and counters</h2>
    <div class="sems" id="sems" style="margin-top:10px"></div>
    <div class="stats" id="stats"></div>
    <p class="note" id="syncNote"></p>
  </div>
  <div class="panel"><h2>Event log</h2><ul id="log" aria-live="polite"></ul></div>
</section>

<section class="panel">
  <h2>The algorithm</h2>
  <div class="code2" style="margin-top:10px">
<pre>// Shared: empty = N, full = 0, mutex = 1
Producer (student):
  loop:
    request = writeRequest()
    wait(empty)    // free slot on board?
    wait(mutex)    // lock the board
    board[in] = request
    in = (in + 1) % N
    signal(mutex)
    signal(full)   // one more request</pre>
<pre>Consumer (staff):
  loop:
    wait(full)     // any request waiting?
    wait(mutex)    // lock the board
    request = board[out]
    out = (out + 1) % N
    signal(mutex)
    signal(empty)  // one slot freed
    cleanRoom(request)</pre>
  </div>
  <p class="note">Turn semaphores off to watch the three classic failures: overflow (a request overwrites an unread
      one), underflow (staff find nothing to take) and race conditions (two actors use the same pointer at once).</p>
</section>
</div>

<script>
(function(){
const $=id=>document.getElementById(id);
const NAMES=['Aarav','Diya','Rohan','Meera','Kabir'],STAFF=['Raju','Sunita','Ganesh','Lata','Imran'];
const TASKS=['Sweep','Mop floor','Clean bathroom','Empty bin','Dust shelves','Change linen'];
let st,gen=0,paused=true,started=false;
const rnd=(a,b)=>a+Math.random()*(b-a);
const speed=()=>+$('Sp').value;
const cfg=()=>({N:+$('N').value,P:+$('P').value,C:+$('C').value,pr:+$('Pr').value*100,cr:+$('Cr').value*100,sync:$('sy
      nc').checked});

class Sem{constructor(n,v){this.n=n;this.v=v;this.q=[]}
```

```
 wait(){if(this.v>0){this.v--;return Promise.resolve()}return new Promise(r=>this.q.push(r))}
 signal(){if(this.q.length)this.q.shift()();else this.v++}}

function sleep(ms,g){return new Promise(res=>{let left=ms;const
      t=()=>{if(g!==gen)return;if(!paused)left-=50*speed();if(left<=0)res();else setTimeout(t,50)};t()})}

function log(msg,cls){const li=document.createElement('li');li.className=cls||'';li.textContent=msg;const
      L=$('log');L.prepend(li);while(L.children.length>80)L.lastChild.remove()}
function set(a,state,note){a.state=state;a.note=note;render()}

function init(){
  gen++;started=false;paused=true;$('go').textContent='Start';
  const c=cfg();
  st={c,slots:Array(c.N).fill(null),in:0,out:0,inCS:0,empty:new Sem('empty',c.N),full:new Sem('full',0),mutex:new
      Sem('mutex',1),
    made:0,done:0,over:0,under:0,race:0,nextId:1,
    prods:Array.from({length:c.P},(_,i)=>({name:NAMES[i],state:'idle',note:'Ready'})),
    cons:Array.from({length:c.C},(_,i)=>({name:STAFF[i],state:'idle',note:'Ready'}))};
  $('log').innerHTML='';
  $('syncNote').textContent=c.sync?'Synchronised: no overflow, underflow or races are possible.':'Unsynchronised:
      errors will appear in red.';
  render();
}

async function enter(a,g){ // critical section marker, detects overlap when no mutex
  st.inCS++;if(st.inCS>1){st.race++;log('Race condition: two actors touching the board at once ('+a.name+')','e')}
}
async function producer(a,g){
  while(g===gen){
    set(a,'working','Writing a request?');await sleep(rnd(.6,1.4)*st.c.pr,g);if(g!==gen)return;
    const item={id:st.nextId++,room:Math.floor(rnd(1,5))*100+Math.floor(rnd(1,25)),task:TASKS[Math.floor(rnd(0,TASKS.l
      ength))],by:a.name};
    if(st.c.sync){
      set(a,'blocked','Blocked on wait(empty): board full');if(!st.empty.v)log(a.name+' waits: board is full','w');
      await st.empty.wait();if(g!==gen)return;
      set(a,'blocked','Blocked on wait(mutex)');await st.mutex.wait();if(g!==gen)return;
    }
    set(a,'cs','Posting R'+item.id+' (room '+item.room+')');enter(a,g);
    const idx=st.in;await sleep(500,g);if(g!==gen)return;
    if(st.slots[idx]){st.over++;log('Overflow: '+a.name+' overwrote unread request R'+st.slots[idx].id,'e')}
    st.slots[idx]=item;st.in=(idx+1)%st.c.N;st.made++;st.inCS--;
    log(a.name+' posted R'+item.id+': '+item.task+', room '+item.room,'p');
    if(st.c.sync){st.mutex.signal();st.full.signal()}
    render();
  }
}
async function consumer(a,g){
  while(g===gen){
    if(st.c.sync){
      set(a,'blocked','Blocked on wait(full): no requests');if(!st.full.v)log(a.name+' waits: board is empty','w');
      await st.full.wait();if(g!==gen)return;
      set(a,'blocked','Blocked on wait(mutex)');await st.mutex.wait();if(g!==gen)return;
    } else {set(a,'working','Checking the board?');await sleep(rnd(.5,1)*st.c.cr/2,g);if(g!==gen)return}
    set(a,'cs','Taking a request');enter(a,g);
    const idx=st.out;await sleep(500,g);if(g!==gen)return;
    const item=st.slots[idx];st.inCS--;
    if(!item){st.under++;log('Underflow: '+a.name+' found an empty slot','e');set(a,'err','Underflow: nothing to
      take');if(st.c.sync){st.mutex.signal();st.empty.signal()}
      await sleep(600,g);continue}
    st.slots[idx]=null;st.out=(idx+1)%st.c.N;
    if(st.c.sync){st.mutex.signal();st.empty.signal()}
    log(a.name+' took R'+item.id+' and is cleaning room '+item.room,'c');
    set(a,'working','Cleaning room '+item.room+' ('+item.task+')');
    await sleep(rnd(.7,1.3)*st.c.cr,g);if(g!==gen)return;
    st.done++;log('Room '+item.room+' cleaned by '+a.name,'c');render();
  }
}

function render(){
  if(!st)return;const c=st.c,lab={idle:'idle',working:'working',blocked:'waiting',cs:'in critical
      section',err:'error'};
  const actorHTML=a=>`<div class="actor ${a.state}"><div><b>${a.name}</b><small>${a.note}</small></div><span
      class="chip">${lab[a.state]}</span></div>`;
  $('prods').innerHTML=st.prods.map(actorHTML).join('');
  $('cons').innerHTML=st.cons.map(actorHTML).join('');
  $('board').innerHTML=st.slots.map((s,i)=>`<div class="slot ${s?'full':''}"><span class="idx">slot
      ${i}</span>${s?`<div class="tag">R${s.id} · Room ${s.room}<br>${s.task}</div>`:'<div
      class="empty">free</div>'}<div class="ptr">${i===st.in?'<span class="in">in</span>':''}${i===st.out?'<span
      class="out">out</span>':''}</div></div>`).join('');
  const sv=n=>c.sync?st[n].v:'?',wq=n=>c.sync?st[n].q.length+' waiting':'not used';
  $('sems').innerHTML=['empty','full','mutex'].map(n=>`<div
      class="sem"><span>${n}</span><b>${sv(n)}</b><span>${wq(n)}</span></div>`).join('');
  const occ=st.slots.filter(Boolean).length;

      $('stats').innerHTML=`<div><b>${st.made}</b>posted</div><div><b>${st.done}</b>cleaned</div><div><b>${occ}/${c.N}
```

Hostel Cleaning Desk: Producer-Consumer Simulator (hostel-producer-consumer.html)

```
      </b>on board</div><div class="bad"><b>${st.over+st.under+st.race}</b>errors</div>`;
}

function start(){
  if(!started){started=true;const g=gen;st.prods.forEach(a=>producer(a,g));st.cons.forEach(a=>consumer(a,g))}
  paused=!paused;$('go').textContent=paused?'Resume':'Pause';
}
$('go').onclick=start;$('reset').onclick=init;
['N','P','C','sync'].forEach(i=>$(i).addEventListener('change',init));
const labels={N:['vN',v=>v],P:['vP',v=>v],C:['vC',v=>v],Pr:['vPr',v=>(v/10).toFixed(1)+'
      s'],Cr:['vCr',v=>(v/10).toFixed(1)+' s'],Sp:['vSp',v=>v+'×']};
Object.keys(labels).forEach(k=>{const
      f=()=>{$(labels[k][0]).textContent=labels[k][1]($(k).value)};$(k).addEventListener('input',f);f();
  if(k==='Pr'||k==='Cr')$(k).addEventListener('input',()=>{if(st){st.c.pr=+$('Pr').value*100;st.c.cr=+$('Cr').value*10
      0}})});
$('presetFull').onclick=()=>{$('P').value=4;$('C').value=1;$('Pr').value=15;$('Cr').value=60;$('N').value=4;sync();};
$('presetEmpty').onclick=()=>{$('P').value=1;$('C').value=4;$('Pr').value=60;$('Cr').value=15;$('N').value=5;sync();};
function sync(){Object.keys(labels).forEach(k=>$(k).dispatchEvent(new Event('input')));init()}
$('theme').onclick=()=>{const r=document.documentElement,d=getComputedStyle(r).getPropertyValue('--
      bg').trim()==='#12181c';r.dataset.theme=d?'light':'dark'};
init();
})();
</script>
</body>
</html>
```